PANEL INTERVIEW · AI SOLUTIONS ENGINEER · LLNL

Production-Grade RAG & LLM Systems

Architecting for deterministic outcomes, governance & trust

Anshul Sharma

June 10, 2026

Anshul Sharma

Software Engineering · **AI / GenAI Architecture**

Staff Software Engineer / Technical Lead · M.S. Software Engineering · 13 years of industry experience
Author of *RAG in Production* · Inventor of the **ILEI Architecture for LLMs** (USPTO Provisional 63/076,477)

Distributed Systems
scale · reliability · failure modes

Productivity Apps
product sense · UX at scale · fast iteration

Agentic Systems
tool use · orchestration · control loops

RAG / GenAI
retrieval · evaluation · guardrails

The lens I bring

- I don't see a model — I see a **distributed system with a probabilistic component** in the critical path
- **Failure modes first** — what breaks, when, and why — before the happy path
- Non-determinism is **contained and measured**, not hand-waved
- "Good" is **defined in measurable terms** before we build

Operating principle

*"Treat the LLM as the **least reliable component** in the system — and engineer everything around containing and measuring that uncertainty."*

Everyone starts here — the naive RAG

THE DEMO PIPELINE — ONE STRAIGHT LINE

Documents → Fixed-size chunk → Embed → Vector DB → Top-k retrieve → LLM → Answer

- ✕ **Fixed chunking** splits at arbitrary boundaries — tables mid-row, code mid-block, conclusions cut from their caveats
- ✕ **Pure vector search** misses exact terms — part numbers, acronyms
- ✕ **No re-ranking** — right doc buried below noise
- ✕ **No access control** — retrieves chunks the user can't see
- ✕ **No evaluation** — hallucinations ship silently
- ✕ **No guardrails / citations** — confident, unsupported answers

Works in a 20-minute demo. Fails the moment it meets real data, real users, and real consequences.

What production actually requires — three planes

KNOWLEDGE PLANE — OFFLINE, GOVERNED, VERSIONED, REPRODUCIBLE

Sources
docs · DBs · APIs
→
Ingest / ETL
parse · clean · dedupe
→
Structure-aware chunking
tables · code · sections
→
Embedding
model-versioned
→

Vector + Metadata store
indexed · filterable

*Each chunk carries: source · section · version · **access-control identity** · PII tags · lineage*

SERVING PLANE — ONLINE, LOW-LATENCY

Query
→
Input guardrails
injection · PII · scope
→
Rewrite / expand
+ embed
→
Hybrid retrieve
dense + BM25 · ACL filter
→
Re-rank
cross-encoder
→

Context assemble
dedupe · token budget
→
LLM
grounded prompt
→
Output guardrails
faithfulness · citations · schema
→
Answer + citations

*ACLs enforced **at retrieval time** · **refuse-over-guess** on low-confidence retrieval · latency budget per stage*

CONTROL PLANE — TRUST, GOVERNANCE & OPERATIONS, SPANNING BOTH PLANES

Evaluation harness
faithfulness · precision · deploy gate

Observability / tracing
spans · latency · cost

Access & governance
identity · ACLs · audit log

Caching & cost controls
semantic cache · budgets

Retrieval is the system. The LLM is the last 10%.

Retrieval — the real lever

- **Hybrid search:** dense vectors + sparse BM25 — pure semantic quietly fails on part numbers, acronyms, exact terms
- **Re-ranking** fixes ordering — consistently beats a bigger, costlier model
- **Structure-aware chunking** — tables, code blocks, and conclusions stay whole
- When it hallucinates, my first question is "show me the retrieved context" — not "upgrade the model"

Evaluation — the deployment gate

- **Retrieval:** context **precision** & **recall** — right evidence, low noise
- **Generation: faithfulness** — is every claim grounded in retrieved context?
- **Answer relevance** — did we address the actual question?
- Curated eval set built **with domain experts**; run on every change; **gate deploys on it like a test suite**

> *"A fluent, confident, unsupported answer is the most dangerous output a RAG system can produce. I'd rather it say 'I don't have enough information' than guess — refusal is a feature, not a failure."*

3 · MOST RECENT ENTERPRISE DATA + AI PROJECT

Unified knowledge assistant — Wiki · Slack · Jira

Engineers lost hours hunting answers fragmented across three systems.

Solution: one grounded assistant — but each source fails differently and needed its own ingestion strategy.

THREE SOURCES · THREE DIFFERENT FAILURE MODES

Wiki / Docs

Challenge: goes stale, deeply nested

Fix: recency decay · section-aware chunking

Slack (public channels)

Challenge: conversational noise, no ground truth

Fix: thread reconstruction · **public-only ACL filter**

Jira Tickets

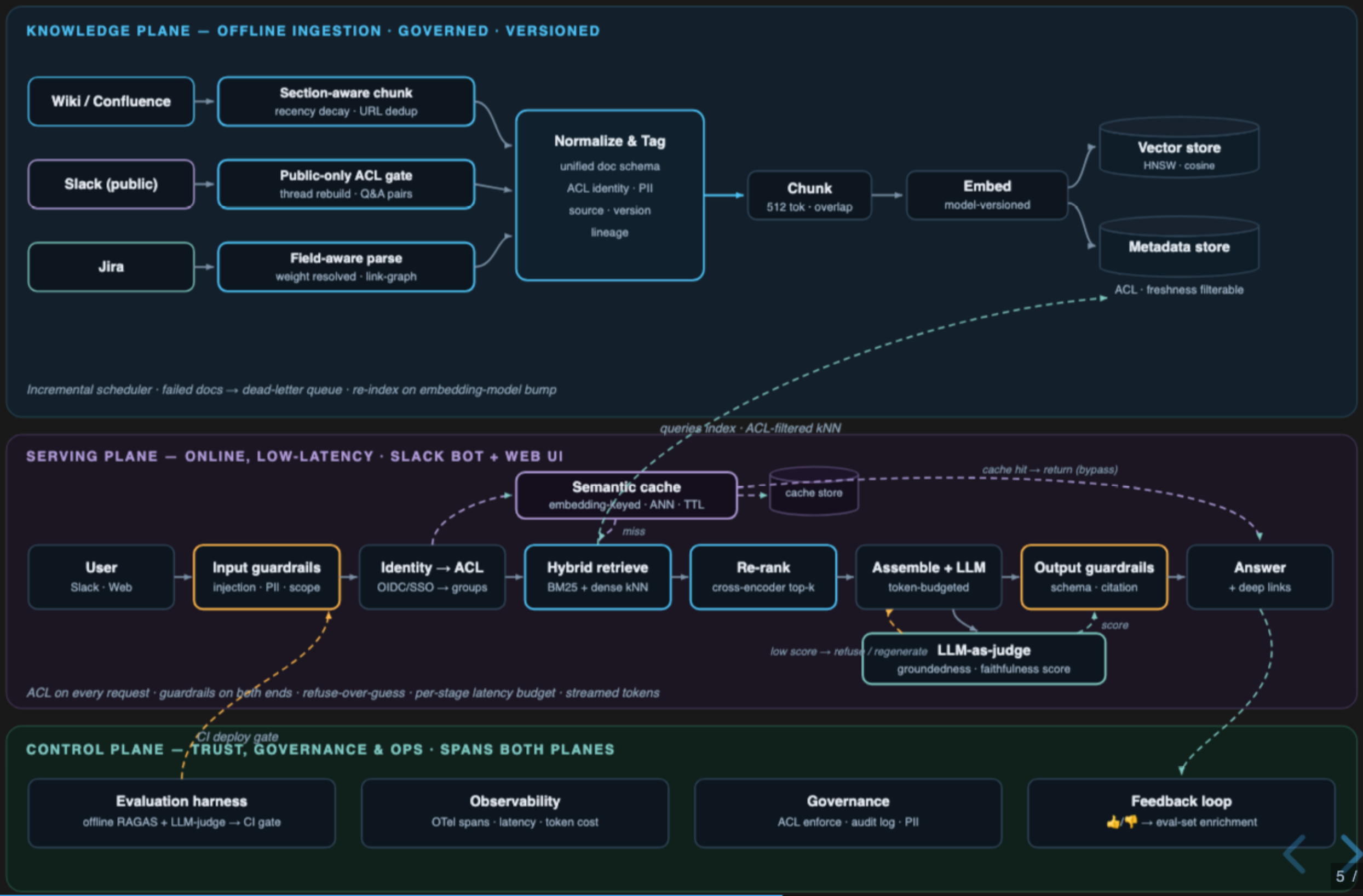
Challenge: structured fields mixed with freeform comments

Fix: field-aware parsing · weight resolved answers

The hard part wasn't the LLM — it was that each source breaks differently, and they all had to converge into one access-controlled index.

3 · MOST RECENT ENTERPRISE DATA + AI PROJECT

Production architecture — end-to-end



3 · MOST RECENT ENTERPRISE DATA + AI PROJECT — IMPACT

IMPACT

↓80%	30,000+	0.92	3 → 1
SEARCH TIME	MAN-HOURS SAVED / YEAR	FAITHFULNESS (EVAL SET)	SYSTEMS TO SEARCH

WHAT I LEARNED

- Per-source ingestion strategy matters more than the LLM choice — **each source breaks differently**
- **Hybrid retrieval + re-ranking** beat upgrading the model every time
- An **eval harness built with SMEs** is the only way to ship with confidence
- Meet users where they are — **Slack-native answers with deep links** drove adoption

Tradeoff I'd make again

Chose **precision over coverage** — tuned to **refuse-over-guess**, so a confident wrong answer was near-zero in eval. We accepted more "not enough information" responses to protect trust in the early rollout.

What changed the team

Aligned **security, data owners & SMEs** on access policy before build; the eval gate moved the team from "looks good" to **measured**.

Opinionated, but loyal to capabilities, not vendors

Layer	What I use / evaluate	How I decide
Models	Claude (Opus/Sonnet/Haiku), OpenAI (GPT-4o, GPT-5.x), open-weight (Llama, Mistral) for in-boundary/self-host	Capability per $ & latency; can it run inside the security boundary?
Orchestration	LangChain / LlamaIndex for speed; **thin custom layer** when control matters	Frameworks to prototype; own the critical path in production
Retrieval / store	pgvector, Milvus/Qdrant; BM25/Elastic for hybrid; Azure AI Search	Scale, filtering on ACL metadata, ops burden
Evaluation	RAGAS-style metrics, custom harnesses, LLM-as-judge (calibrated)	Must gate CI; measure retrieval & generation separately
Guardrails	Structured output / schema validation, input & output filters, citation enforcement; Llama Guard, Presidio	Refuse-over-guess; every claim traceable
Ops	Tracing (OpenTelemetry / LangSmith-style), prompt & index versioning, caching; self-hosted Langfuse	Observability and reproducibility are non-negotiable

> *"I adopt frameworks to move fast, but I own the critical path. In a deterministic-outcomes environment, I won't hand reliability to a black box I can't trace or test."*

CLOSE

The goal shouldn't be a working demo —
it should be an LLM system the organization can defend, audit, and trust at scale.

—

Thank you. I'd love to dig into any of these.